F1 Statistics API: Technical Report

COMP3011 Coursework 1: Kyriacos Konis

Repository: https://github.com/kyrikonis/F1_API
Deployment: https://kyrik2005.pythonanywhere.com
Docs: https://github.com/kyrikonis/F1_API/blob/main/api_documentation.pdf
Presentation: API_Presentation.pptx

Introduction

This project is a RESTful API for Formula 1 race data ranging from 1950 to 2024. You can query drivers, teams, races, and results, alongside further analytics for championship standings, leaderboards, and driver career statistics.

Technology Stack

Python was the natural choice for me given my past development experience. I primarily chose FastAPI In particular for its quality-of-life benefits. FastAPI's automatic OpenAPI integration generates an interactive Swagger UI at /docs without any additional code needed, which I used as my primary manual testing tool throughout the development process. Pydantic's built-in validation also removed the need for any manual input checking, allowing FastAPI to reject any bad requests automatically.

For me SQLite was also a natural fit for this project. I considered PostgreSQL, however it requires a separate server process which adds unnecessary complexity for a project of this scope. Since this API is naturally read-heavy and all write operations are restricted to authenticated users, SQLite's single-writer limitation is not a major concern. Its serverless, single-file format also made it simple to version-control and deploy directly alongside the application.
SQLAlchemy was chosen as writing raw SQL for every query becomes difficult to maintain, especially as the tables naturally have many relationships between them. SQLAlchemy's ORM handles this easily by allowing me to interact with the database through Python rather than SQL.

Architecture and Design

Router-Only Architecture

I considered implementing a service layer between the routers and the database, as it's commonly recommended in FastAPI projects, and I evaluated the trade-offs using AI assistance. I decided against it as the majority of endpoints perform straight forward CRUD operations on existing static data without any meaningful business logic, and adding service and repository layers introduced no clear benefit for a project of this scope.
Instead, the complexity is managed through single-purpose modules: **utils.py** handles reusable identifier resolution logic, Pydantic schemas centralise the response and requests alongside handling validation automatically, and **auth.py** holds all the security logic.
This kept the repository easily readable and was easy to maintain throughout development.

Multiple Identifier Types

For usability, I made the decision to allow multiple identifier types rather than the only option being the internal database IDs. **Utils.py** resolves queries by ID, three-letter driver code, or name, so **/drivers/HAM, /drivers/Hamilton**, and **/drivers/1** all return the same result. This makes the API much more intuitive to use, especially for users who aren't experienced with the database already. I explored this decision with the assistance of AI, evaluating the trade-off between implementation and user convenience.

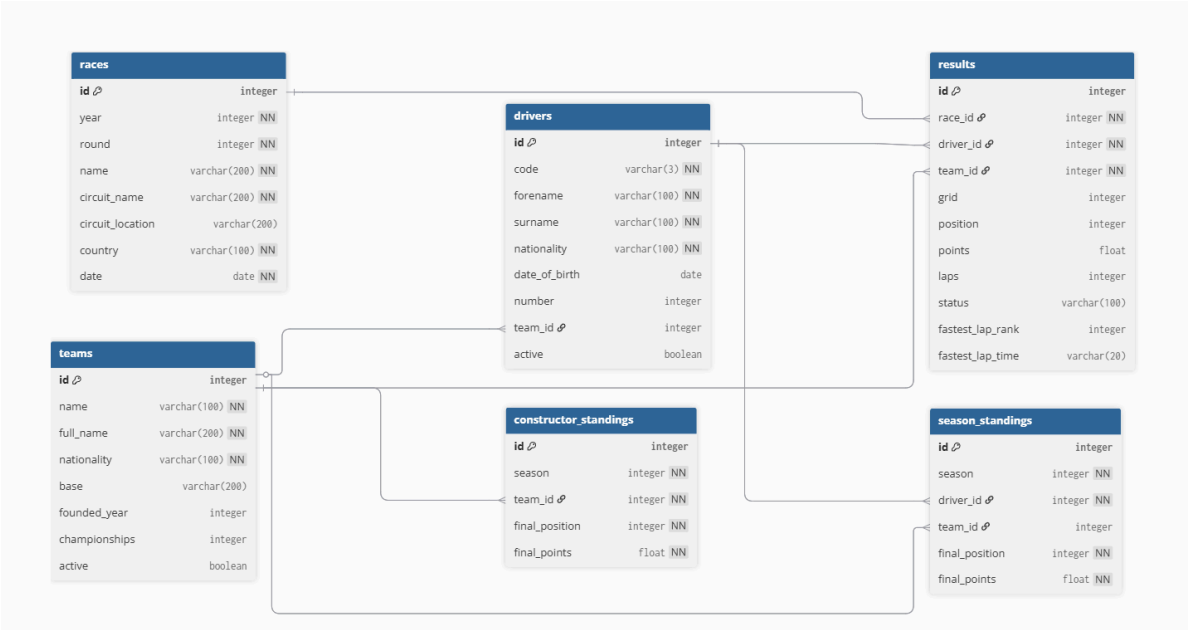
Analysis Endpoints

Beyond standard CRUD, I implemented a set of analytical endpoints that made good use of SQLAlchemy's querying capabilities. **SeasonStanding** and **ConstructorStanding** models read from official championship data rather than aggregating the results, ensuring complete accuracy. I decided this due to the complexity of F1's scoring system and its various rule changes and edge cases throughout history, where aggregating results directly would produce inaccurate standings for many seasons.
The **analysis/leaderboard/wins** endpoint demonstrates aggregation using **func.count()**, while **analysis/drivers/{ref}/career** goes through multiple relationships to return in-detail career statistics in a single query, including total races, wins, podiums, and number of finishes with points.

Database Design

The schema follows a normalised relational model built around **Drivers**, **Teams**, **Races**, and **Results**. The results table acts as a junction between the other three, where it links a driver and team to a specific race alongside their performance data. As stated previously there are also **SeasonStanding** and **ConstructorStanding** models which store official championship data. A source field ('verified'/'live') is also returned in driver and constructor standings responses so consumers can know whether the data is from official records ('verified') or from aggregated results ('live').

Figure 1: Entity-relationship diagram of the F1 API database schema



One issue I ran into during implementation was that I had initially set driver numbers as globally unique across the database. This caused issues as permanent driver numbers weren't introduced until 2014, so numbers were freely reused across seasons before then.
The database was seeded using the Ergast Motor Racing Dataset from Kaggle, which covers around 900 drivers, 300+ teams, and over 1,100 races ranging from 1950 to 2024.
The initial **seed.py** format was generated with AI assistance to handle the bulk of the CSV importing using Pandas, which covered reading the files and mapping columns to model fields. I refined the script to account for inserting teams before drivers, and handle edge cases in the data such as an na() helper for null values, code deduplication for drivers sharing a code, and more data quality inconsistencies.

Security and Deployment

Authentication

Write operations are protected behind an API Key to prevent unauthorised modifications of the database. My initial implementation had two security weaknesses in a hardcoded fallback key, and a basic string comparison for validation. After reading further into FastAPI's security documentation I realised my implementation was prone to timing attacks, where an attacker can infer a valid key by measuring the differences in comparison time.
I also evaluated whether to hash the stored API key for additional security. Since the key is stored as an environment variable rather than in the database, hashing it would offer no real benefit as if an attacker already has access to the server's environment, the key itself is no longer a major concern.
The hardcoded fallback was removed so the application now fails to startup if **API_KEY** is not set, and the comparison was replaced with **secrets.compare_digest()** for constant-time evaluation, as recommended in the FastAPI security documentation.

Deployment

I deployed my API to PythonAnywhere as it offers free Python web application hosting which is more than sufficient for this API and it requires little configuration.
However, two issues arose which did not appear when running the API locally. The SQLite database path broke in production because PythonAnywhere runs the app from a different working directory, which was fixed by deriving an absolute path from __file__ rather than relying on a relative one. I also spent time building a WSGI wrapper which turned out to be unnecessary as PythonAnywhere can run Uvicorn directly through its pa website create command. After I resolved those issues, the deployment was straightforward and I continued to develop locally by pushing to GitHub, pulling on the server, and reloading the site for any further changes.

Testing

The test structure uses pytest alongside FastAPI's built in TestClient, which allows HTTP requests to be made without running a live server, keeping the tests fast and self-contained. All 41 tests run against a fresh in-memory SQLite database created per session, ensuring the testing does not affect the main database.
Tests are split across five files (test_drivers.py, test_teams.py, test_races.py, test_results.py, and test_analysis.py), with a shared conftest.py to handle database setup and the test client.

The initial structure of the test files and conftest.py was generated with AI assistance, which I adapted to use an in-memory SQLite database per session.The tests cover: list endpoints with query filters, GET by different identifiers, creation with response validation, duplicate rejection, foreign key violations, PATCH and DELETE operations, and the analytical endpoints for standings, all-time win leaderboards and career summaries.

One consequence of requiring API_KEY at startup is that the tests also require the variable to be set, since auth.py is imported when the application loads. Running python -m pytest without the environment variable fails immediately before any test executes.

Reflections and challenges

The most significant lesson I learnt during this project was how important domain knowledge is when data modelling, and not just technical knowledge. Although I already follow F1, there were a large number of historical rule changes and edge cases in many of the assumptions I made which I hadn't questioned. Assuming driver numbers to be unique is one example of this, as permanent numbers were only introduced in 2014 and prior to that they were reassigned every season. Similarly, aggregating points to derive championship standings ignores dropped scores and the countback system for tiebreakers which have occurred numerous times throughout F1 history, which is why official standings are now stored separately and used where available, with aggregated results only as a fallback for seasons outside the dataset.

Another challenge I didn't anticipate was how carefully I had to handle the results data in comparison to the other models. A finishing position cannot always be assigned as a driver who has retired from a race has no position, so that had to be nullable. Points also needed to be a float rather than an integer, since half-points have actually been awarded in races which have been stopped before 75% of completion. A bonus point being given for the fastest lap in more recent seasons of the dataset was also something to keep in mind. These all seem like small details at first, but getting them wrong could not only lead to inaccuracies, but potentially corrupted or missing data during seeding. The duplicate result condition was another problem, as the same driver could appear twice in a single race if there were data inconsistencies without a check in the router, which could cause even more issues when aggregating statistics.

Limitations and Future Development

The current authentication relies on a single shared API key, configured via the API_KEY environment variable. Although this provides sufficient protection for this project's scope it has clear limitations, there's no per-user identity, no token expiry, and changing the key requires restarting the server since it is read at startup. To make the authentication more complete, it could use JWT-based authentication with role-based access control, separating the admin permissions properly from read-only.
SQLite works well given the workload is read-heavy, however it doesn't support concurrent writes which is a major limitation for real-world production environments with simultaneous users. Moving to PostgreSQL would resolve this issue entirely, and due to SQLAlchemy's abstraction layer, it would only require minor changes in the code.

The dataset is also static, seeded from the Ergast Motor Racing Dataset up to 2024. Currently there's no way for new race results to automatically update, so future improvement would be to integrate a live data source such as the OpenF1 API which is widely used and provides all the data this project requires.
The analysis endpoints currently support only season standings and career summaries. There's significant scope to expand here with additional analytics such as head-to-head driver comparisons, circuit-specific statistics, and points progression charts across a season for both the Drivers' and Constructors' championships.
Finally, the API has no rate limiting, so there is nothing there to prevent a user from flooding the server with requests. Request throttling per IP address would be simple to add and definitely a worthwhile improvement before a more public deployment.

References

fastapi.tiangolo.com (2024) *FastAPI Security Documentation*. Available at: https://fastapi.tiangolo.com/advanced/security/http-basic-auth/#fix-it-with-secrets-compare-digest

docs.sqlalchemy.org (2024) *SQLAlchemy 2.0 Documentation*. Available at: https://docs.sqlalchemy.org

rohanrao (2024) *Formula 1 World Championship (1950–2024)*. Kaggle. Available at: https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020?resource=download

PythonAnywhere (2024) *Deploying a FastAPI App*. Available at: https://help.pythonanywhere.com/pages/FastAPI/

OpenF1 (2024) *OpenF1 API*. Available at: https://openf1.org

Appendix - Gen AI Declaration

Generative AI was used throughout this project via Perplexity AI to support design exploration, planning the implementation, debugging, documentation, and deployment troubleshooting. A large part of the AI usage was to compare alternatives and evaluate pros and cons, while all final implementation decisions remained my responsibility.

Chat Log 1 supported the project from the start through to testing and deployment.
Its most valuable use was in exploring design decisions rather than only generating code. For example, after it recommended a service layer, I used it to evaluate whether a service layer was justified over router-based logic for this project in particular, which helped me weigh out the advantages and disadvantages, before deciding to keep the current router-only structure.
Another example is during seed script development, the initial AI-generated import approach used db.add(), which would provide duplicate insertion errors when repeatedly run against an existing database. I identified this issue and recommended switching the seeding logic to db.merge() instead, which let the script be re-run safely. This was appropriate as the seed data will always be the same due to the static dataset, so updating existing rows was the behaviour I required.
I also utilised AI when working through the dataset and working around edge cases. For example, for reusing driver codes in the Ergast dataset and comparing possible ways of ensuring accuracy while keeping the API practical to use.
Separate chat logs were also used for README formatting and PythonAnywhere deployment troubleshooting. These tasks were quite simple, but they still saved time and helped improve the quality of the repository, and help fix deployment issues.

Chat Log 1

2. README Formatting
Guidance on structuring the README file, including how to correctly present the tech stack, installation instructions, and tables within the README such as for the API endpoints.

Chat Log 2

3. PythonAnywhere Deployment Troubleshooting
Resolving a uWSGI worker hang, it provided a solution by dropping wsgi.py and deploying via PythonAnywhere's pa website create command with Uvicorn directly instead.

Chat Log 3

Example Screenshots from Chat Log 1:

I'm building the SQLAlchemy ORM models now: Team/Constructor , Driver, Result, and I was also thinking Race.

For Driver I want there to be one showing if they are active as I want the API not to just pull from the current grid, as well as statistics on their number of championships, alongside other necessary ones you seem fitting.

All models are quite closely linked, so Race would probably show what season it was in, the circuit name / country etc, Whilst Result would link the driver to the race and show their starting and finishing position, points for the championship scored etc.

What fields do you think would be necessary in this case for each field on top of the ones I've listed and do you think the separate models for Result and Race make sense, or should it be aggregated?

Show less

Completed 1 step

Keeping `Race` and `Result` as separate models is the right call — they model genuinely different things. `Race` is an **event** (happened once, fixed metadata), while `Result` is a **participation record** (one row per driver per race). Merging them would denormalise badly, repeating circuit/date data across every driver's result row.

Suggested fields per model

Team / Constructor

```python
class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str                          # e.g. "Red Bull Racing"
    short_name: str                    # e.g. "Red Bull"
    nationality: str
    base: str                          # e.g. "Milton Keynes, UK"
    constructor_ref: str               # Ergast-style slug e.g. "red_bull"
    first_entry_year: int | None
    championships: int = Field(default=0)
    is_active: bool = Field(default=True)
```

Driver

```python
class Driver(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    forename: str
    surname: str
```

Ask a follow-up

Claude Sonnet 4.6 Thinking

Would having championships, race wins, total points etc stored within DriveResponse not lead to the data at times being inaccurate unless manually updated?

Completed 1 step

Yes, exactly — that's the core tradeoff of denormalisation. Those fields on the `Driver` model (not just the response) would drift out of sync any time a `Result` row is inserted, updated, or deleted without also updating the driver's cached stats.

The two approaches

Denormalised (what was suggested): Store `race_wins`, `total_points` etc. directly on `Driver`. Fast reads, but you're responsible for keeping them consistent — every result write needs a corresponding driver stat update. If you forget once, the data is wrong silently.

Computed on the fly: Don't store them on `Driver` at all. `DriverResponse` aggregates them via a DB query at request time using `SUM`, `COUNT` etc. across `Result` rows. Always accurate, slightly slower, but for an F1 dataset (at most ~1000 drivers, ~20k result rows ever) the query cost is trivial.

What makes more sense here

Since your dataset is relatively small and mostly static (historical F1 data doesn't change), computing from `Result` is the cleaner approach. Your `DriverResponse` would then be populated in the route rather than mapping directly from the ORM model:

```python
class DriverResponse(DriverBase):
    id: int
    team_id: int | None
    championships: int        # queried/counted, not stored on Driver
    race_wins: int
    total_points: float
    ...
```

The `Driver` ORM model itself only stores what's a true persistent attribute of the driver — `championships` could stay since it's not derivable from your `Result` table (it's a season-level accolade), but `race_wins`, `total_points`, `podiums` are all perfectly derivable from `Result` rows and shouldn't need to live on the model.

Prepared using Claude Sonnet 4.6

So could I not do:

class DriverBase(BaseModel):
code: ...
forename: ...

Show more

Completed 1 step

Yes, exactly right. That's the cleanest approach. `DriverResponse` just adds `id` and anything computed, and your route handler does the aggregation work.

One thing to flag on your championships calculation — you can't purely derive a championship from "most points in a season" in all historical cases. There are edge cases like:

- **1974, 1984** — points were equal or near-equal but the championship was decided by countback rules (most wins, then most second places, etc.)

Ask a follow-up

Claude Sonnet 4.6 Thinking

What are the pros/cons of keeping it in the router vs a service layer, since this is a small scale api project

Completed 1 step

For a small-scale project the difference is minimal in practice — it's more about code organisation than any technical necessity.

Keeping it in the router

Pros:

- Less files, less indirection — everything for a route is in one place
- Faster to write, easier to trace when debugging
- Perfectly acceptable for coursework/portfolio scale

Cons:

- Routers get long quickly once you add analytical endpoints
- If you ever want to reuse logic (e.g. the same FK validation in both POST and PATCH), you'd be duplicating it
- Harder to unit test — you'd have to mock HTTP requests rather than just calling a function directly

Service layer

Pros:

- Business logic is independently testable without spinning up a router
- Reusable — `create_result_service` could be called by a bulk-import endpoint later too
- Cleaner separation makes the codebase easier to present and explain
- Scales naturally if the project grows

Cons:

- Extra files and an extra call hop for simple operations
- Can feel like over-engineering when the "service" is just three lines

Practical take for your project

Given you're presenting this, the service layer is worth it **selectively** — only where logic is genuinely non-trivial. `Result` POST is a clear candidate as already discussed. Simple CRUD on `Race`, `Driver`, `Team` doesn't need it.

The hybrid approach is the most common real-world pattern too — not everything lives in a service, only the parts that benefit from it.

Prepared using Claude Sonnet 4.6

Since it's only three db.get() checks I've kept it in the router as the scale at this stage isn't enough to warrant a service layer in my opinion.

```
@router.post("/", response_model=ResultResponse, status_code=201)
def create_result(payload: ResultCreate, db: Session =
Depends(get_database)):
if not db.get(Race, payload.race_id):
raise HTTPException(status_code=404, detail=f"Race {payload.race_id} not
found")
if not db.get(Driver, payload.driver_id):
raise HTTPException(status_c =404, detail=f"Driver {payload.driver_id} not
```

Ask a follow-up

Claude Sonnet 4.6 Thinking

My seed.py is failing with

UNIQUE constraint failed: drivers.code when importing the CSV data.

My driver model has code = Column(String(3), unique=True).

I done some research and driver codes and they weren't permanent before 2014 and often reused across different drivers so what's the right way to handle this? I've seen I can relax the constraint on the model but It could lead to further errors, what would be the pros and cons of different solutions?
I don't really want to result to fallback codes for current drivers, and would prefer prioritising the latest drivers to have their correct driver codes

Show less

Completed 1 step

Good research — this is a genuine data quality issue with the Ergast dataset. Here are your realistic options:

Option 1 — Remove `unique=True`, add a partial unique index

Keep codes stored as-is but only enforce uniqueness for active drivers. SQLAlchemy supports this with a partial index:

```python
from sqlalchemy import Index

code = Column(String(3), nullable=True)

__table_args__ = (
    Index(
        "uq_driver_code_active",
        "code",
        unique=True,
        postgresql_where=text("is_active = true"),    # Postgres
        # sqlite_where=text("is_active = 1"),         # SQLite
    ),
)
```

Pros: Historically accurate, no data loss, modern drivers keep their real codes, old reused codes coexist cleanly

Cons: SQLite's partial index support is limited — the `WHERE` clause works on SQLite 3.8.9+ but SQLAlchemy doesn't always generate it correctly for SQLite, so you'd be relying on application-level validation rather than the DB constraint for historical rows